

LAB3 예비 보고서

FPGA 응용회로 사전 시뮬레이션 — PWM·구동기·표시장치·직렬통신

과목 전자전기컴퓨터설계실험 2

제출일 2026년 9월 28일

성명 김태이

학번 2023440158

1 보고서 범위와 결론

이 보고서는 LAB3의 필수 실험 7개(교안 번호 19-25)를 대상으로 회로 목적, 블록 흐름, 파라미터 계산, 상태·타이밍 표, RTL·TB·XDC의 역할, TB 자극과 기대 결과, 정상 시뮬레이션 결과, 한 항목 수정 실험과 복구 결과, 보드에서 확인할 항목을 정리한다.

7개 정상 설계 모두 Icarus Verilog에서 교안이 제시한 PASS 문자열과 검사 수를 확인했고, Vivado XSim에서도 같은 PASS를 얻었다. 수정 실험은 실험마다 교안이 지정한 한 항목만 바꿨다. 01·02·03·05·06은 TB가 첫 불일치에서 \$fatal로 멈췄고, 04·07은 순서가 유지되는 수정이라 PASS가 나며 시간 간격만 바뀌었다. 모두 원본으로 복구한 뒤 정상 PASS를 다시 확인했다.

PASS는 TB가 지정한 디지털 비교를 통과했다는 뜻이다. 핀 배선, 전기 규격, 부품의 물리 동작은 포함하지 않으므로 각 장 끝의 보드 확인 항목에서 따로 확인한다. 7개 설계는 Vivado에서 합성·구현·bitstream 생성까지 마쳤다 (DRC 오류 0, WNS 양수).

실험	정상 결과	수정 실험 결과
01 PWM LED	checks=4 · 3831 ns	LEVELS 10→5: FAIL high=6
02 RGB PWM	checks=2 · 4431 ns	R 초기 30 %: FAIL 5,5,8
03 피에조	edges=5 · 551 ns	TONE ×1.25: FAIL 반주기 4
04 스텝모터	checks=8 · 831 ns	STEP_HZ 절반: PASS, 1311 ns
05 MM:SS	checks=7 · 144031 ns	CLK_HZ 2→4: FAIL 00:05
06 문자 LCD	bytes=40 · 8130 ns	R→S: FAIL data=53
07 UART	checks=3 · 9970 ns	BAUD ×2: PASS, 5170 ns

표의 종료 시각은 Icarus 기준이다. XSim은 07만 9910 ns로 다르며 원인은 LAB3-07 장에 적었다.

2 실험 환경과 검증 방법

각 실험은 fpga-lab-template v2.0.2를 새 폴더로 받아 교안의 RTL·TB·XDC·simulation.json을 입력해 만들었다. VS Code 작업의 01 Check tools, 02 Simulate와 같은 tools/fpga_lab.py로 Icarus Verilog 12(iverilog -g2012 -Wall, vvp)를 실행했고, 생성된 wave.vcd를 VaporView로 열어 확인했다. 컴파일 경고는 없었다.

교차 확인으로 같은 RTL·TB를 Vivado 2025.2 XSim에서도 실행했다. 교안은 Vivado 2026.1 기준이며, 이 PC에는 2025.2가 설치되어 있다. 구현은 Part xc7s75fpga484-1, 설계 top은 lab3_*, 시뮬레이션 top은 tb_*로 구분했다.

TB 클록은 #10 반전으로 주기 20 ns, 즉 보드의 50 MHz와 같다. 대신 CLK_HZ 같은 시간 상수를 작은 값으로 덮어써 실제 순서와 경계 조건은 유지하면서 실행 시간을 줄인다. 파형의 시간은 보드 시간과 다르므로 각 장의 파라미터 표에 두 값을 함께 적었다.

3 공통 설계 규칙

모든 순차 로직은 MAIN CLOCK F(B6)의 clk_50mhz 하나만 쓴다. PWM 주기, 모터 step, 1초, LCD tick, UART 비트 시간은 카운터가 만드는 clock-enable로 처리하고, 분주한 신호를 다른 always 블록의 클록으로 쓰지 않는다. 버튼·UART RX 같은 비동기 입력은 두 플립 플롭으로 동기화한 뒤 사용한다.

기능	기준 계산
PWM 주기	CLK_HZ / PWM_HZ
피에조 반주기	CLK_HZ / (2 · TONE_HZ)
모터 step 간격	CLK_HZ / STEP_HZ

기능	기준 계산
1초 tick	CLK_HZ 클록마다 1회
UART bit	반올림한 CLK_HZ / BAUD

모든 XDC는 top 포트 전부에 PACKAGE_PIN과 LVCMOS33을 지정하고, clk_50mhz에 20.000 ns create_clock을 건다. 비동기 입력(rst_p, 버튼, UART RX)에는 false path를 둔다.

4 소스 출처와 실행 스냅샷

항목	식별 정보
템플릿 원격	https://github.com/Glaysia/fpga-lab-template.git
기준 태그-커밋	v2.0.2 · 0b3318e466e9d77f7d3880df09c827788f361913
상위 저장소	https://github.com/tae2yi/UOS_ece2
LAB2 기준 커밋	67b9370 (lab2_3)
LAB3 코드 커밋	090e0da (보고서의 GitHub 주소가 가리키는 커밋)
실행 폴더	lab3/lab3_01_led_pwm ... lab3_07_uart_echo

GitHub: https://github.com/tae2yi/UOS_ece2/tree/090e0da/lab3

LAB3의 RTL·TB·XDC, 빌드 스크립트, 수정 실험 자료는 커밋 090e0da에 들어 있다. 각 코드 블록 아래의 GitHub 주소는 이 커밋에 고정되어 해당 줄 범위를 가리키므로, 이후 파일이 바뀌어도 보고서의 코드와 일치한다. 파일 역할 표의 파일 이름도 같은 커밋의 파일로 연결된다. 실험 후 보고서에는 최종 commit hash를 다시 적는다.

LAB3-01 PWM LED 밝기

회로 목적과 블록 흐름

버튼을 한 번 누를 때마다 PWM duty를 10 %씩 올리고, 100 % 다음에는 0 %로 돌아가게 한다. 8개 LED에 같은 PWM 신호를 내보내므로 모든 LED의 밝기가 함께 바뀐다.

버튼(N8)은 FPGA 클록과 무관한 시점에 바뀌므로 button_onepulse가 두 플립플롭으로 먼저 동기화한다. 동기화된 값이 이전에 인정한 값(accepted)과 다른 상태가 STABLE_CYCLES(20 ms) 동안 이어져야 새 값으로 인정하고, 놀림으로 바뀔 때만 한 클록 폭의 press를 낸다. top은 press마다 level을 0에서 10까지 올리고 10 다음에 0으로 되돌린다. pwm_channel은 0~49,999를 도는 count와 threshold를 비교해 pwm을 만들고, top이 이를 led[7:0]에 8번 복제한다. 모든 레지스터는 clk_50mhz 하나로 동작하며 느린 동작은 카운터로 만든다.

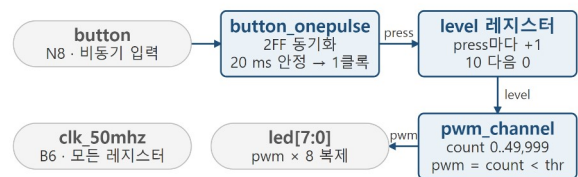


그림 1. LAB3-01 블록 흐름. 설계 top은 lab3_led_pwm.

파라미터 계산

항목	식	보드 (50 MHz)	TB
PWM 주기	CLK_HZ / PWM_HZ	50,000클록 = 1 ms	10클록
count 폭	clog2(주기)	16 bit	4 bit
threshold	주기 × level / LEVELS	5,000 × level	level
디바운스	STABLE_CYCLES	1,000,000클록 = 20 ms	2클록
단계 수	LEVELS + 1	11단계 (0~100 %)	11단계

TB는 CLK_HZ=1000, PWM_HZ=100, DEBOUNCE_CYCLES=2로 덮어써서 한 주기를 10클록으로 줄인다. 비율(level/LEVELS)은 보드와 같으므로 HIGH 클록 수가 곧 level이 된다.

level별 threshold와 duty

level	threshold (보드)	duty	TB HIGH/10
0	0	0 %	0
1	5,000	10 %	1
3	15,000	30 %	3
5	25,000	50 %	5
9	45,000	90 %	9
10	50,000	100 %	10
10 → 누름	0	0 %	0

RTL·TB·XDC의 역할

파일	역할
src/button_onepulse.v	2FF 동기화, 20 ms 안정 확인, 놀림당 1클록 press

파일	역할
src/pwm_channel.v	주기 카운터와 비교기, 레지스터 출력 pwm
src/lab3_led_pwm.v	설계 top. level 레지스터, 하위 모듈 연결, led 8비트 복제
sim/tb_led_pwm.sv	시뮬레이션 top. 가속 파라미터로 DUT를 구동하고 한 주기 HIGH 수를 검사
constraints/lab3_led_pwm.xdc	B6-K4-N8-LED 8핀, LVCMOS33, 20 ns create_clock, rst_p-button false path

pwm_channel.v의 threshold 계산과 PWM 레지스터 부분이다.

```
always @* begin
    if (level >= LEVELS)
        threshold = PERIOD_CYCLES;
    else
        threshold = (PERIOD_CYCLES * level) / LEVELS;
end
always @(posedge clk or posedge rst_p) begin
    if (rst_p) begin
        count <= {COUNT_WIDTH{1'b0}};
        pwm <= 1'b0;
    end else begin
        pwm <= (count < threshold);
        if (count == PERIOD_CYCLES - 1)
            count <= {COUNT_WIDTH{1'b0}};
        else
            count <= count + 1'b1;
    end
end
end
```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_01_led_pwm/src/pwm_channel.v#L15-L33

level이 LEVELS 이상이면 threshold를 주기 전체로 두어 100 %를 보장한다. pwm은 비교 결과를 한 클록 늦게 내보내는 레지스터 출력이라 클리치가 없다.

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p=1 3클록 후 해제	level=0, HIGH 0
2	버튼 3회	level=3, HIGH 3
3	버튼 7회 추가	level=10, HIGH 10
4	버튼 1회 추가	level=0 순환, HIGH 0

한 번 누르기는 button=1 6클록, 0 6클록이다. 2FF 지연 2클록과 DEBOUNCE_CYCLES=2를 넘기는 길이다. expect_level은 count=0이 되는 순간부터 10클록 동안 led[0]의 HIGH 수를 세고, 다르면 \$fatal로 멈춘다.

Icarus PASS와 정상 파형

```
$ python tools/fpga_lab.py simulate
LAB3_LED_PWM_PASS checks=4
sim/tb_led_pwm.sv:31: $finish called at 3831000 (1ps)
```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_01_led_pwm_icarus_normal.txt

Icarus 종료 3831 ns, XSim 종료 3831 ns. 버튼 11회 입력에 따라 led가 00과 ff 사이를 오가며, 단계가 올라갈수록 HIGH 구간이 길어진다. 네 번의 검사(0 %, 30 %, 100 %, 다시 0 %)가 모두 통과했다.

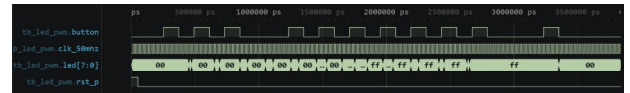


그림 2. LAB3-01 정상 실행 전체 파형 (VaporView). 버튼 입력마다 led의 HIGH 폭이 늘어난다.

수정 실험과 복구 결과

교안의 수정 실험은 LEVELS를 10에서 5로 바꾸는 것이다. TB가 LEVELS를 인스턴스에서 덮어쓰므로 TB 인스턴스의 값을 바꿨다. RTL 기본값만 바꾸면 시뮬레이션에 반영되지 않는다. 기대값은 그대로 둔다.

```
-lab3_led_pwm #(...,LEVELS(10),...) dut
+lab3_led_pwm #(...,LEVELS(5),...) dut
```

diff:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/mod/01_led_pwm/modification.diff

구분	내용
예상	버튼 3회 → level=3 → threshold = 10×3/5 = 6 → HIGH 6. 기대 3과 달라 FAIL
수정 결과	\$fatal level=3 high=6 (1231 ns). threshold가 0→2→4→6으로 20 %씩 증가
복구 결과	LEVELS=10 → HIGH 3 → LAB3_LED_PWM_PASS checks=4 (3831 ns)

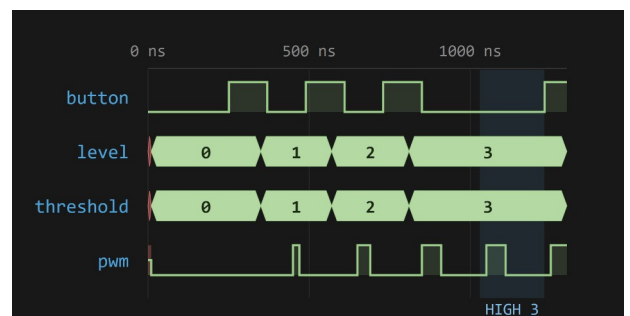


그림 3. 정상(LEVELS=10): 측정 구간 10클록 중 pwm HIGH 3클록.

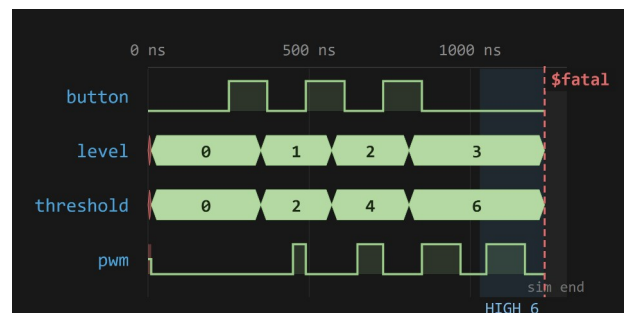


그림 4. 수정(LEVELS=5): 같은 구간에서 HIGH 6클록, 1231 ns에 \$fatal.

보드 기준으로는 level 3의 threshold가 15,000(30 %)에서 30,000(60 %)으로 바뀐다. 단계도 11단계(10 %씩)에서 6단계(20 %씩)로 줄어 버튼 6번이면 한 바퀴를 돈다.

보드에서 확인할 항목

확인 항목	예상 관찰
리셋(K4)	모든 LED 꺼짐 (level 0)
버튼 1회	한 단계만 밝아짐. 두 단계 이상 켜면 디바운스 확인

확인 항목	예상 관찰
버튼 10회	최대 밝기 (100 %)
11번째 누름	다시 꺼짐 (0 %)
깜빡임	1 kHz라 보이지 않아야 함. 오실로스코프로 1 ms 주기와 duty 확인
극성	LED가 active-low면 밝기 순서가 반대로 보임. 보드 회로도 확인

LAB3-02 RGB LED PWM

회로 목적과 블록 흐름

R·G·B 버튼 세 개로 세 색의 duty를 각각 0~100 % (10 % 단위)로 바꾸고, 네 개의 RGB LED에 같은 색을 낸다. 세 채널은 주기가 같아서 혼합색이 duty 비율로 정해진다.

LAB3-01의 입력 처리와 PWM 채널을 색마다 하나씩, 모두 세 벌 둔다. button_onepulse 세 개가 버튼별 press를 만들고 level_r·level_g·level_b가 서로 독립적으로 0→10→0을 순환한다. pwm_channel 세 개는 같은 PERIOD_CYCLES=50,000을 쓰므로 세 PWM이 같은 순간(count=0)에 켜지고 각자의 threshold에서 꺼진다. 각 pwm은 4비트로 복제되어 led_r·led_g·led_b[3:0]로 나간다.



그림 5. LAB3-02 블록 흐름. 설계 top은 lab3_rgb_pwm.

파라미터 계산

항목	식	보드 (50 MHz)	TB
PWM 주기	CLK_HZ / PWM_HZ	50,000클록 = 1 ms	10클록
threshold	주기 × level / 10	채널마다 5,000 × level	level
디바운스	STABLE_CYCLES	20 ms (버튼마다)	2클록
색 조합	11 × 11 × 11	1,331가지	—

세 채널이 같은 주기와 위상을 쓰므로 한 주기 안에서 R·G·B가 켜져 있는 시간 비율이 곧 각 색의 세기다.

레벨 조합과 예상 색

(R, G, B) level	예상 색
(10, 0, 0)	빨강
(0, 10, 0)	초록
(0, 0, 10)	파랑

(R, G, B) level	예상 색
(10, 10, 0)	노랑
(0, 10, 10)	청록
(10, 0, 10)	자홍
(10, 10, 10)	흰색
(2, 5, 8)	TB 검사값. 파랑이 강한 하늘색

RTL·TB·XDC의 역할

파일	역할
src/button_onepulse.v	LAB3-01과 같음. 버튼마다 한 개씩
src/pwm_channel.v	LAB3-01과 같음. 색마다 한 개씩
src/lab3_rgb_pwm.v	설계 top. 원펄스·level·PWM 세 벌과 4비트 복제
sim/tb_rgb_pwm.sv	시뮬레이션 top. 세 색의 HIGH 수를 동시에 측정
constraints/lab3_rgb_pwm.xdc	버튼 N8/N4/N1, led_r T2·U1·P2·R3, led_g U5·V1·R7·T6, led_b U3·W2·R5·T3

lab3_rgb_pwm.v의 레벨 레지스터 부분이다. 세 조건문이 독립이라 여러 버튼이 같은 클록에 눌러도 각자 갱신된다.

```

always @(posedge clk_50mhz or posedge rst_p) begin
  if (rst_p) begin
    level_r <= 0; level_g <= 0; level_b <= 0;
  end else begin
    if (press_r)
      level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;
    if (press_g)
      level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;
    if (press_b)
      level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
  end
end
  
```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_02_rgb_pwm/src/lab3_rgb_pwm.v#L28-L36

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p 해제	세 레벨 0
2	R 2회, G 5회, B 8회	HIGH 2 / 5 / 8
3	R 1회 추가	HIGH 3 / 5 / 8 (G·B 유지)

measure는 R 채널 count가 0이 된 뒤 10클록 동안 세 색의 HIGH 수를 함께 센다. 마지막 검사는 R 버튼이 G·B 레벨을 건드리지 않는다는 채널 독립성을 확인한다.

Icarus PASS와 정상 파형

```

$ python tools/fpga_lab.py simulate
LAB3_RGB_PWM_PASS checks=2
sim/tb_rgb_pwm.sv:26: $finish called at 4431000 (1ps)
  
```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_02_rgb_pwm_icarus_normal.txt

Icarus 종료 4431 ns, XSim 종료 4431 ns. 파형에서 button_r 2회, button_g 5회, button_b 8회 뒤 마지막에 button_r이 한 번 더 눌린다. 두 번의 측정이 모두 통과했다.

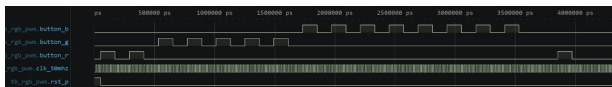


그림 6. LAB3-02 정상 실행 전체 파형 (VaporView). R·G·B 버튼 입력 순서가 보인다.

수정 실험과 복구 결과

교안은 한 색의 초기 duty만 바꾸도록 한다. level_r의 리셋 값을 0에서 3(30 %)으로 바꾸고 TB는 그대로 둔다.

```
-level_r <= 0;    level_g <= 0; level_b <= 0;
+level_r <= 4'd3; level_g <= 0; level_b <= 0;
```

diff:

https://github.com/tae2yi/UOS_ee2/blob/090e0da/lab3/report/pre/mod/02_rgb_pwm/modification.diff

구분	내용
예상	ROI 3에서 2회 증가 → level_r=5 → HIGH 5/5/8. 기대 2/5/8과 달라 FAIL
수정 결과	\$fatal rgb high=5,5,8 (3851 ns). pwm_r 폭이 pwm_g와 같아짐
복구 결과	LAB3_RGB_PWM_PASS checks=2 (4431 ns)



그림 7. 정상: 측정 구간에서 R·G·B HIGH 2·5·8클록.



그림 8. 수정(R 초기 30 %): R·G·B HIGH 5·5·8클록, 3851 ns에 \$fatal.

혼합색은 (20, 50, 80) %의 하늘색에서 (50, 50, 80) %로 R이 더해져 연보라 쪽으로 이동한다. 보드에서는 리셋 직후부터 R만 30 %로 켜진다.

보드에서 확인할 항목

확인 항목	예상 관찰
리셋	네 RGB LED 모두 꺼짐
버튼별 입력	누른 버튼의 색만 한 단계 밝아짐
세 색 10	흰색에 가까운 색. 한 색이 약하면 해당 핀 확인
네 LED	모두 같은 색. 하나만 다르다면 그 LED의 XDC 핀 확인
극성	공통 애노드형이면 LOW에서 켜져 밝기가 반대로 보임

LAB3-03 피에조 단일 음

회로 목적과 블록 흐름

50 MHz 클록을 분주해 294 Hz(D4, 레) 사각파를 만들고 피에조로 한 음을 계속 낸다.

입력은 리셋뿐이다. count가 0~HALF_PERIOD-1을 돌고 끝 값에 닿을 때마다 piezo를 반전한다. 반전 두 번이 한 주기이므로 출력 주파수는 $\text{CLK_HZ} / (2 \times \text{HALF_PERIOD})$ 이고 duty는 50 %다. 분주한 신호를 다른 블록의 클록으로 쓰지 않고, piezo는 출력 핀으로만 나간다.

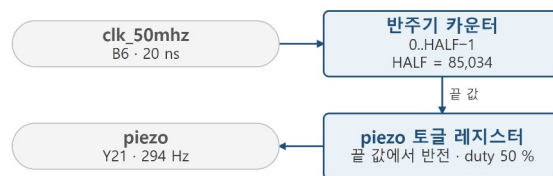


그림 9. LAB3-03 블록 흐름. 설계 top은 lab3_piezo.

파라미터 계산

항목	식	보드 (50 MHz)	TB
반주기	$\text{CLK_HZ} / (2 \cdot \text{TONE_HZ})$	85,034클록	5클록
count 폭	$\text{clog2}(\text{반주기})$	17 bit	3 bit
실제 주파수	$\text{CLK_HZ} / (2 \cdot \text{반주기})$	294.000 Hz	100 Hz(가속)
주기	$2 \times \text{반주기} \times 20 \text{ ns}$	3.401 ms	200 ns

정수 나눗셈 때문에 반주기는 85,034.01에서 85,034로 내림되지만 주파수 오차는 0.001 %보다 작다.

음 높이별 분주값 (CLK_HZ = 50 MHz)

음	TONE_HZ	반주기	실제 주파수
C4 도	262	95,419	262.002 Hz
D4 레	294	85,034	294.000 Hz
E4 미	330	75,757	330.003 Hz

음	TONE_HZ	반주기	실제 주파수
F4 파	349	71,633	349.001 Hz
G4 솔	392	63,775	392.003 Hz
A4 라	440	56,818	440.001 Hz
B4 시	494	50,607	494.003 Hz

RTL·TB·XDC의 역할

파일	역할
src/lab3_piezo.v	설계 top. 반주기 카운터와 토글 레지스터
sim/tb_piezo.sv	시뮬레이션 top. count=0 간격이 정확히 5 클록인지 검사
constraints/lab3_piezo.xdc	B6-K4-Y21, LVCMOS33, 20 ns create_clock, rst_p false path

lab3_piezo.v의 분주 부분 전체다.

```
localparam integer HALF_PERIOD = CLK_HZ / (2 * TONE_HZ);
always @(posedge clk_50mhz or posedge rst_p) begin
  if (rst_p) begin
    count <= 0;
    piezo <= 0;
  end else if (count == HALF_PERIOD - 1) begin
    count <= 0;
    piezo <= ~piezo;
  end else begin
    count <= count + 1'b1;
  end
end
```

GitHub:

https://github.com/tae2yi/UOS_ee2/blob/090e0da/lab3/lab3_03_piezo/src/lab3_piezo.v#L10-L24

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p 3클록 후 해제	piezo=0, count=0
2	25클록 관찰	count=0 간격이 매번 5클록
3	관찰 종료	반전 4회 이상 → edges 출력

TB는 CLK_HZ=1000, TONE_HZ=100으로 반주기를 5 클록으로 줄인다. 매 클록 piezo에 X가 없는지도 확인한다.

Icarus PASS와 정상 파형

```
$ python tools/fpga_lab.py simulate
LAB3_PIEZO_PASS edges=5
sim/tb_piezo.sv:20: $finish called at 551000 (1ps)
```

로그:

https://github.com/tae2yi/UOS_ee2/blob/090e0da/lab3/report/pre/logs/lab3_03_piezo_icarus_normal.txt

Icarus 종료 551 ns, XSim 종료 551 ns. 파형에서 리셋 해제 뒤 piezo가 5클록(100 ns)마다 반전하는 50 % 사각파를 확인했다.



그림 10. LAB3-03 정상 실행 전체 파형 (VaporView). piezo가 100 ns마다 반전한다.

수정 실험과 복구 결과

교안은 TONE_HZ를 다른 음으로 바꾸도록 한다. TB는 가속 파라미터를 쓰므로 음 높이 비율을 1.25배로 줄여 TB 인스턴스의 TONE_HZ를 100에서 125로 바꿨다. 294 Hz(D4)를 370 Hz(F#4)로 바꾸는 것과 비율이 거의 같다.

```
-lab3_piezo #(.CLK_HZ(1000),.TONE_HZ(100)) dut(...);
+lab3_piezo #(.CLK_HZ(1000),.TONE_HZ(125)) dut(...);
```

diff:

https://github.com/tae2yi/UOS_ee2/blob/090e0da/lab3/report/pre/mod/03_piezo/modification.diff

구분	내용
예상	반주기 = 1000/(2×125) = 4클록. 기대 5와 달라 FAIL
수정 결과	\$fatal half period=4 (191 ns). count가 0~3만 순환
복구 결과	LAB3_PIEZO_PASS edges=5 (551 ns)

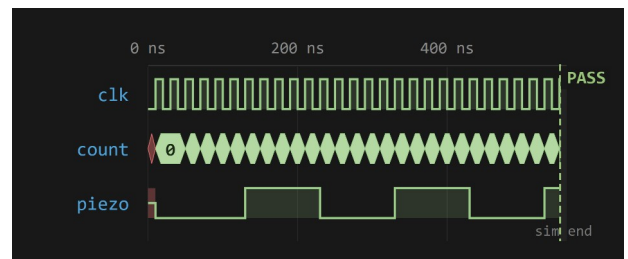


그림 11. 정상(TONE_HZ=100): count 0~4, 100 ns마다 반전.

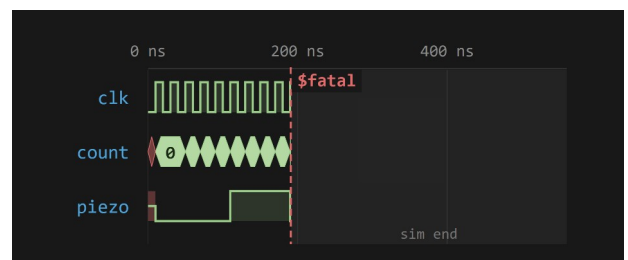


그림 12. 수정(TONE_HZ=125): count 0~3, 80 ns마다 반전, 191 ns에 \$fatal.

보드 기준으로 370 Hz는 반주기 67,567클록(370.003 Hz)이다. 분주값이 작아질수록 높은 음이 난다.

보드에서 확인할 항목

확인 항목	예상 관찰
소리	294 Hz 연속음 (레, D4)
리셋 중	무음 (piezo=0 유지)
부품 종류	수동형 피에조여야 함. 능동 부저는 자체 발진음이 섞임
오실로스코프	Y21에서 주기 3.401 ms, duty 50 % 확인
음량	3.3 V 직접 구동이라 작을 수 있음

LAB3-04 스텝모터 위상 제어

회로 목적과 블록 흐름

clock-enable마다 4상 코일 패턴을 한 칸씩 이동해 스텝 모터를 정회전·역회전하고, enable로 정지시킨다.

enable(N8)과 direction(N4)은 레벨 입력이지만 비동기이므로 각각 2FF로 동기화한다. enable_sync가 1인 동안 step 카운터가 STEP_CYCLES마다 한 번 끝 값에 닿고, 그때 state를 direction_sync에 따라 +1 또는 -1 한다. 조합 디코더가 2비트 state를 두 코일이 동시에 켜지는 4비트 패턴으로 바꾼다. enable_sync가 0이면 카운터만 0으로 돌아가고 state는 유지된다.

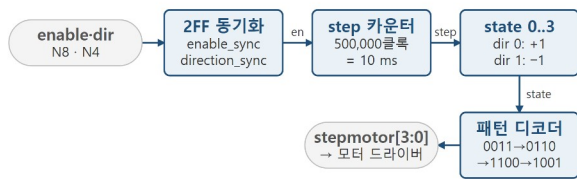


그림 13. LAB3-04 블록 흐름. 설계 top은 lab3_stepper.

파라미터 계산

항목	식	보드 (50 MHz)	TB
step 간격	CLK_HZ / STEP_HZ	500,000클럭 = 10 ms	4클럭
count 폭	$\log_2(\text{간격})$	19 bit	2 bit
step 속도	STEP_HZ	100 step/s	—
회전 속도	$\text{STEP_HZ} \times 60 / \text{회전당 스텝}$	200스텝 모터면 30 rpm	—

회전당 스텝 수는 모터에 따라 다르다(예: 1.8° 모터 200 스텝). 실험 모터의 사양을 보고 rpm을 다시 계산한다.

state와 코일 패턴

state	stepmotor[3:0]	dir=0 다음	dir=1 다음
0	0011	1	3
1	0110	2	0
2	1100	3	1
3	1001	0	2

RTL·TB·XDC의 역할

파일	역할
src/lab3_stepper.v	설계 top. 입력 동기화, step 카운터, state, 패턴 디코더
sim/tb_stepper.sv	시뮬레이션 top. 정방향 한 주기, 역방향 2단계, 정지 유지 검사
constraints/lab3_stepper.xdc	N8·N4, stepmotor[3:0] Y20·Y22·AA20·AA21, 20 ns create_clock

lab3_stepper.v의 step 카운터와 state 갱신 부분이다.

```
always @(posedge clk_50mhz or posedge rst_p) begin
  if (rst_p) begin
    count <= 0;
    state <= 0;
  end else if (!enable_sync) begin
    count <= 0;
  end else if (count == STEP_CYCLES - 1) begin
    count <= 0;
    state <= direction_sync ? state - 1'b1 : state + 1'b1;
  end else begin
    count <= count + 1'b1;
  end
end
```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_04_stepper/src/lab3_stepper.v#L29-L41

2비트 state의 ±1은 자연스럽게 0→3을 순환하므로 별도 경계 처리가 필요 없다.

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p 해제	0011
2	enable=1	0110 → 1100 → 1001 → 0011
3	direction=1	1001 → 1100
4	enable=0, 12클럭 대기	1100 유지

TB는 CLK_HZ=8, STEP_HZ=2로 간격을 4클럭으로 줄인다. dut.state가 다음 값이 될 때까지 기다린 뒤 1 ns 후 stepmotor를 비교하므로 순서를 검사하고, 간격 자체는 파형으로 확인한다.

Icarus PASS와 정상 파형

```
$ python tools/fpga_lab.py simulate
LAB3_STEPPER_PASS checks=8
sim/tb_stepper.sv:20: $finish called at 831000 (1ps)
```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_04_stepper_icarus_normal.txt

Icarus 종료 831 ns, XSim 종료 831 ns. 파형에서 enable이 켜진 뒤 4클럭마다 상이 이동하고, direction이 바뀌면 역순으로, enable이 꺼지면 멈춘다.



그림 14. LAB3-04 정상 실행 전체 파형 (VaporView). enable·direction 입력 구간이 보인다.

수정 실험과 복구 결과

교안은 STEP_HZ를 절반으로 바꾸도록 한다. TB 인스턴스의 STEP_HZ를 2에서 1로 바꿨다.

```
-lab3_stepper #(.CLK_HZ(8),.STEP_HZ(2)) dut
+lab3_stepper #(.CLK_HZ(8),.STEP_HZ(1)) dut
```

diff:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/mod/04_stepper/modification.diff

구분	내용
예상	간격 8클럭(160 ns). TB는 순서만 보므로 PASS, 종료만 늦어짐

구분	내용
수정 결과	LAB3_STEPPER_PASS checks=8, 종료 831 → 1311 ns, 상 간격 2배
복구 결과	간격 4클럭, LAB3_STEPPER_PASS checks=8 (831 ns)

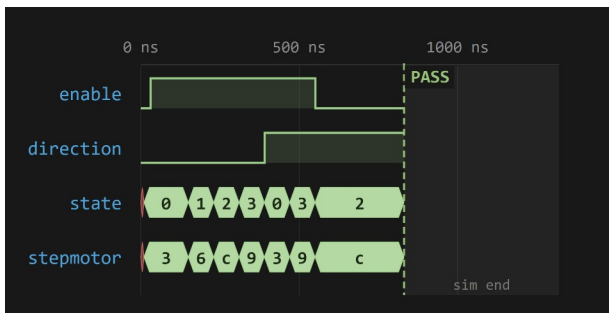


그림 15. 정상(STEP_HZ=2): 80 ns마다 한 상 이동, 831 ns에 PASS. stepmotor는 16진(3=0011, 6=0110, c=1100, 9=1001).

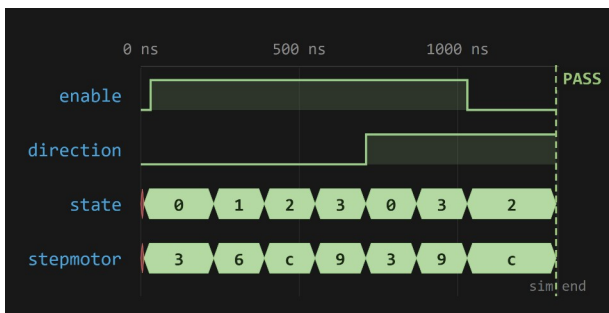


그림 16. 수정(STEP_HZ=1): 같은 순서를 160 ns 간격으로, 1311 ns에 PASS.

보드 기준으로 STEP_HZ 100→50이면 간격이 10 ms에서 20 ms로 늘어 회전 속도가 절반이 된다. 순서가 그대로이므로 FAIL이 나지 않는 것이 정상이며, 변화는 종료 시각과 파형 간격으로 확인한다.

보드에서 확인할 항목

확인 항목	예상 관찰
결선	모터 드라이버 입력에 연결. 별도 모터 전원, 공통 GND
enable	켜면 회전, 끄면 정지. 정지 중에도 마지막 두 코일에 전류가 흐름(발열 주의)
direction	바꾸면 회전 방향 반전
드라이버 LED	4개 중 2개씩 0011→0110→1100→1001 순서로 이동
탈조	떨리기만 하고 돌지 않으면 STEP_HZ를 낮춰 확인

LAB3-05 MM:SS 시계

회로 목적과 블록 흐름

1초 enable로 00:00부터 59:59까지 세고 다시 00:00으로 돌아가는 시계를 만들고, 4자리 7세그먼트에 MM.SS로 표시한다.

mmss_counter는 subsecond가 CLK_HZ-1에 닿을 때마다 한 번 BCD 자리올림을 한다. 초 1의 자리가 9에서 넘치면 초 10의 자리, 59초에서 분 1의 자리, 59분 59초에서 모두 0으로 돌아간다. 네 자리는 sevenseg_decode 네 개로 세그먼트 패턴이 된다. top은 별도의 스캔 카운터로 250 μs마다 한 자리를 선택해 해당 seg_com만 LOW로 내리고 그 자리의 패턴을 seg_data로 낸다. 분 1의 자리에는 dp를 켜서 콜론 대신 점으로 분과 초를 구분한다.

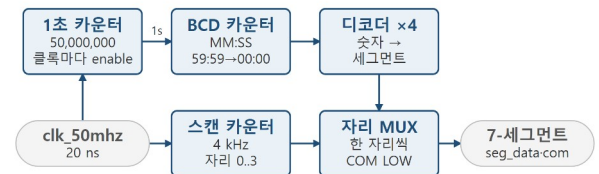


그림 17. LAB3-05 블록 흐름. 설계 top은 lab3_mmss_clock.

파라미터 계산

항목	식	보드 (50 MHz)	TB
1초	CLK_HZ 클럭마다 1 회	50,000,000클럭	2클럭
subsecond 폭	$\log_2(\text{CLK_HZ})$	26 bit	1 bit
스캔 간격	$\text{CLK_HZ} / \text{SCAN_HZ}$	12,500클럭 = 250 μs	—
새로고침	4자리 × 250 μs	1 ms (1 kHz)	—
전체 순환	60 × 60초	3,600초 = 1시간	144 μs

한 자리는 한 번에 250 μs씩 켜져 duty 25 %로 보인다. 새로고침이 1 kHz라 깜빡임이 보이지 않는다.

스캔 순서와 자리 선택 (seg_com은 LOW에서 선택)

scan_select	seg_com	표시 자리
0	1111_0111	분 10의 자리
1	1111_1011	분 1의 자리 + dp
2	1111_1101	초 10의 자리
3	1111_1110	초 1의 자리

RTL·TB·XDC의 역할

파일	역할
src/mmss_counter.v	1초 enable과 BCD 4자리 자리올림
src/sevenseg_decode.v	0~9 → {a,b,c,d,e,f,g,dp} 패턴 (HIGH 점등)
src/lab3_mmss_clock.v	설계 top. 스캔 카운터, 자리 MUX, dp 추가
sim/tb_mmss_counter.sv	시뮬레이션 top. 카운터와 디코더를 직접 검사
constraints/lab3_mmss_clock.xdc	seg_data H2·J7·J3·J1·E4·E2·F5·F1, seg_com K5·K3·K1·L6·G3·G1·H6·H4

mmss_counter.v의 자리올림 부분이다.

```
end else if (subsecond == CLK_HZ - 1) begin
    subsecond <= 0;
    if (second_ones != 9)
        second_ones <= second_ones + 1'b1;
    else begin
        second_ones <= 0;
        if (second_tens != 5)
            second_tens <= second_tens + 1'b1;
        else begin
            second_tens <= 0;
            if (minute_ones != 9)
                minute_ones <= minute_ones + 1'b1;
            else begin
                minute_ones <= 0;
                if (minute_tens != 5)
                    minute_tens <= minute_tens + 1'b1;
                else
                    minute_tens <= 0;
            end
        end
    end
end
end
end
```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_05_mmss_clock/src/mmss_counter.v#L20-L40

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	디코더 0, 9	1111_1100, 1111_0110
2	rst_p 해제	00:00
3	10초	00:10
4	50초 더	01:00
5	3,539초 더	59:59
6	1초 더	00:00

TB는 mmss_counter에 CLK_HZ=2를 주어 1초를 2클록으로 줄이고 advance(n)에서 2n클록을 기다린다. TB가 top을 인스턴스화하지 않으므로 스캔 MUX와 seg_com 극성은 검사 범위 밖이며 보드에서 확인해야 한다.

Icarus PASS와 정상 파형

```
$ python tools/fpga_lab.py simulate
LAB3_MMSS_PASS checks=7
sim/tb_mmss_counter.sv:23: $finish called at 144031000 (1ps)
```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_05_mmss_clock_icarus_normal.txt

Icarus 종료 144031 ns, XSim 종료 144031 ns.
3,600초 전체 순환을 포함해 일곱 검사가 통과했다. 종료 시각 144 μs는 3,600초 × 2클록 × 20 ns와 맞는다.

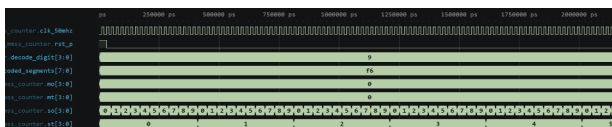


그림 18. LAB3-05 정상 실행 앞부분 파형 (VaporView). so가 0~9를 돌고 st가 한 칸씩 올라간다.

수정 실험과 복구 결과

교안은 TB의 mmss_counter 인스턴스에 전달하는 CLK_HZ를 바꾸도록 한다. TB 클록 20 ns는 그대로 두고 CLK_HZ를 2에서 4로 바꿨다.

```
-mmss_counter #(.CLK_HZ(2)) dut(...);
+mmss_counter #(.CLK_HZ(4)) dut(...);
```

diff:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/mod/05_mmss_clock/modification.diff

구분	내용
예상	1초 = 4클록. advance(10)은 20클록만 기다리므로 00:05 → FAIL
수정 결과	\$fatal time=00:05 (431 ns). 자리올림 간격 2배
복구 결과	LAB3_MMSS_PASS checks=7 (144031 ns)

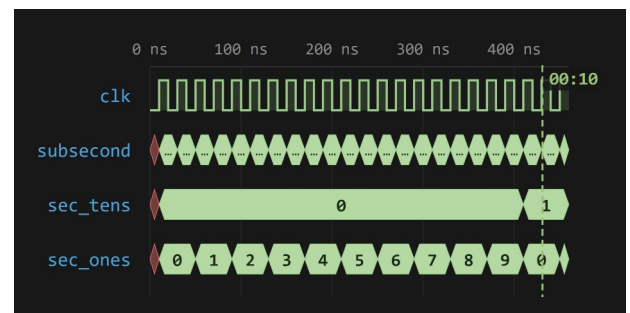


그림 19. 정상(CLK_HZ=2): 40 ns마다 1초, 431 ns에 00:10.

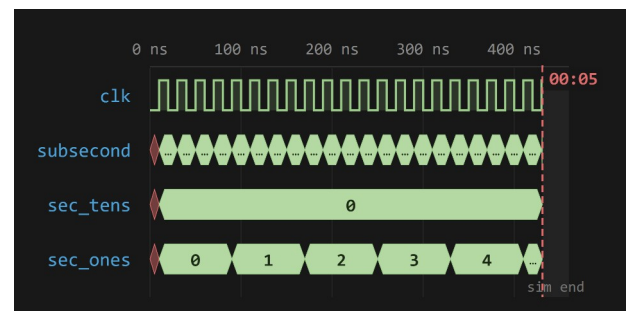


그림 20. 수정(CLK_HZ=4): 80 ns마다 1초, 431 ns에 00:05로 \$fatal.

이 값은 TB 전용이라 bitstream(top CLK_HZ=50,000,000)에는 영향이 없다. 같은 현상을 보드에서 보려면 top의 CLK_HZ를 100,000,000으로 바꾸면 되며, 그러면 1초 표시에 실제 2초가 걸린다.

보드에서 확인할 항목

확인 항목	예상 관찰
시작	리셋 후 00.00 표시
초 증가	1초마다 한 칸. 스톱위치와 1분 이상 비교
자리올림	00.59 → 01.00 (1분 후)
자리-점	분 1의 자리 뒤에 점. 자리 순서가 뒤바뀌면 seg_com 핀 순서 확인
극성	모든 세그먼트가 반전되면 seg_data-seg_com 극성 확인
나머지 4자리	꺼짐 (seg_com[7:4]=1)

LAB3-06 문자 LCD 제어

회로 목적과 블록 흐름

HD44780 호환 문자 LCD를 8비트 write-only 방식으로 초기화하고 1행 "FPGA LAB3", 2행 "LCD CONTROLLER"를 표시한다.

tick 생성기가 500클럭(10 μ s)마다 한 클럭 폭의 tick을 낸다. 전원 투입 후 2,000 tick(20 ms)을 기다린 뒤 phase FSM이 바이트마다 0(RS-데이터 셋업) $\rightarrow$ 1(E HIGH) $\rightarrow$ 2(E LOW) $\rightarrow$ 3(명령별 대기)을 반복한다. 보낸 바이트는 index로 고르는 명령·문자 표에서 나온다. lcd_rw는 0으로 묶어 busy flag를 읽지 않는 대신 고정 대기 시간을 둔다. 마지막 바이트(index 39) 다음에는 index 6(1행 주소)으로 돌아가 같은 내용을 계속 다시 쓴다.

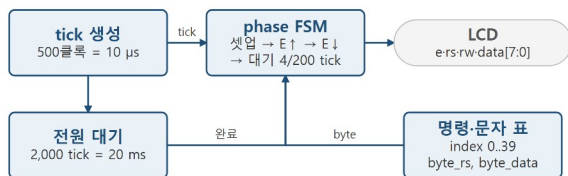


그림 21. LAB3-06 블록 흐름. 설계 top은 lab3_character_lcd.

파라미터 계산

항목	식	보드 (50 MHz)	TB
tick	TICK_CYCLES	500클럭 = 10 μ s	2클럭
전원 대기	POWER_TICKS	2,000 tick = 20 ms	3 tick
E HIGH 폭	1 tick	10 μ s	1 tick
일반 바이트	3 + 4 + 1 tick	8 tick = 80 μ s	5 tick
clear 바이트	3 + 200 + 1 tick	204 tick = 2.04 ms	6 tick
첫 화면	대기 + 40바이트	약 25.2 ms	—

HD44780U 데이터시트 기준 E 펄스 최소 230 ns, 셋업 40 ns, 일반 명령 실행 37 μ s, clear 1.52 ms다. 10 μ s / 80 μ s / 2.04 ms는 모두 여유가 있다. 20 ms 전원 대기도 요구 조건(15 ms 이상)을 만족한다.

전송 바이트 순서 (총 40바이트)

index	RS	데이터	의미
0~2	0	0x38	8비트, 2줄, 5x8 글꼴 (3회)
3	0	0x0C	표시 켜기, 커서 끄
4	0	0x06	주소 자동 증가
5	0	0x01	화면 지우기 (긴 대기)
6	0	0x80	DDRAM 0x00 (1행)
7~22	1	문자	"FPGA LAB3" + 공백 7
23	0	0xC0	DDRAM 0x40 (2행)
24~39	1	문자	"LCD CONTROLLER" + 공백 2

RTL·TB·XDC의 역할

파일	역할
src/lab3_character_lcd.v	설계 top. tick, 전원 대기, phase FSM, 명령·문자 표
sim/tb_character_lcd.sv	시뮬레이션 top. E 하강 에지마다 RS-데이터 40바이트 비교
constraints/lab3_character_lcd.xdc	lcd_e A6, lcd_rs G6, lcd_rw D6, lcd_data A4·B2·C3·D4·A2·C5·C1·D1

lab3_character_lcd.v의 phase FSM 부분이다(전원 대기 이후).

```

case (phase)
0: begin
  lcd_e <= 0; lcd_rs <= byte_rs; lcd_data <= byte_data;
  phase <= 1;
end
1: begin lcd_e <= 1; phase <= 2; end
2: begin
  lcd_e <= 0;
  wait_count <= (index == 5) ? CLEAR_WAIT_TICKS
    : NORMAL_WAIT_TICKS;
  phase <= 3;
end
default: begin
  if (wait_count > 0) wait_count <= wait_count - 1;
  else begin
    index <= (index == 39) ? 6 : index + 1'b1;
    phase <= 0;
  end
end
endcase

```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_06_character_lcd/src/lab3_character_lcd.v#L87-L106

LCD는 E의 하강 에지에서 데이터를 받아들인다. RS-데이터를 E보다 한 tick 먼저 세우고, E가 내려간 뒤 한 tick 동안 유지한다.

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p 3클럭 후 해제	전원 대기 3 tick
2	E 하강 에지 0~6	RS=0, 38·38·38·0C·06·01·80
3	E 하강 에지 7~22	RS=1, "FPGA LAB3" + 공백
4	E 하강 에지 23	RS=0, C0
5	E 하강 에지 24~39	RS=1, "LCD CONTROLLER" + 공백

TB는 lcd_e 하강 에지마다 lcd_rw=0, RS, 데이터를 기대 배열과 비교하고 40바이트가 모두 맞으면 끝난다.

Icarus PASS와 정상 파형

```

$ python tools/fpga_lab.py simulate
LAB3_LCD_PASS bytes=40
sim/tb_character_lcd.sv:31: $finish called at 8130000 (1ps)

```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_06_character_lcd_icarus_normal.txt

Icarus 종료 8130 ns, XSim 종료 8130 ns. 파형에서 lcd_e 펄스가 일정 간격으로 이어지고, lcd_rs가 문자 구간에서 1, 2행 주소 명령(0xC0)에서 잠시 0이 된다.

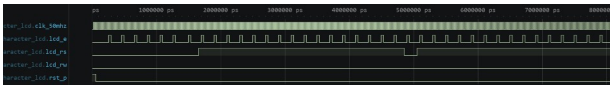


그림 22. LAB3-06 정상 실행 전체 파형 (VaporView). lcd_rs가 0xC0 명령 구간에서만 0으로 내려간다.

수정 실험과 복구 결과

교안은 둘째 줄 문자열 한 글자를 바꾸도록 한다. index 37의 "R"을 "S"로 바꾸고 TB는 그대로 둔다.

```
-37: begin byte_rs=1; byte_data="R"; end
+37: begin byte_rs=1; byte_data="S"; end
```

diff:

https://github.com/tae2yi/UOS_ee2/blob/090e0da/lab3/report/pre/mod/06_character_lcd/modification.diff

구분	내용
예상	"LCD CONTROLLERS". 0x52 → 0x53으로 lcd_data[0]만 변함. 38번째 바이트에서 FAIL
수정 결과	\$fatal index=37 rs=1 data=53 (7730 ns). 앞의 37바이트는 일치
복구 결과	index 37 = R(0x52) → LAB3_LCD_PASS bytes=40 (8130 ns)

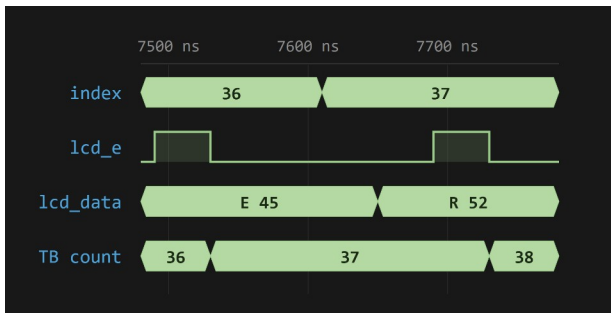


그림 23. 정상: index 37에서 lcd_data = R(0x52).

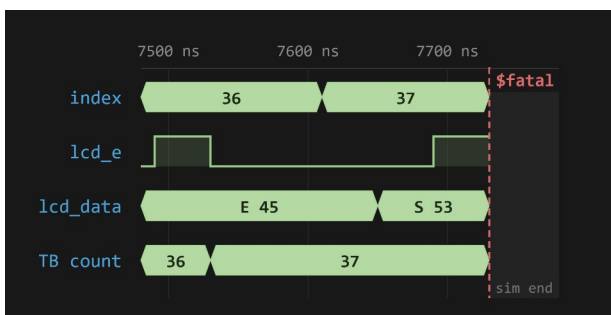


그림 24. 수정: index 37에서 lcd_data = S(0x53), E 하강 에지 (7730 ns)에서 \$fatal.

보드에서는 2행 14번째 칸(DDRAM 0x4D)이 R에서 S로 바뀌어 보인다.

보드에서 확인할 항목

확인 항목	예상 관찰
전원-레벨	LCD 전원(5 V/3.3 V)과 논리 레벨이 FPGA 3.3 V와 맞는지 확인
대비	V0 가변저항 조정. 아무것도 안 보이면 먼저 대비 확인

확인 항목	예상 관찰
표시	1행 "FPGA LAB3", 2행 "LCD CONTROLLER"
초기화 실패	1행에 검은 칸만 보이면 명령 순서-대기 시간-핀 확인
리셋	리셋 후 약 25 ms 안에 다시 표시. 깜빡임 없음

LAB3-07 PC-FPGA UART 에코

회로 목적과 블록 흐름

PC에서 9600 8N1로 받은 한 바이트를 그대로 되돌려 보내고, 마지막 수신값을 LED 8개에 표시한다.

uart_rxd(C6)는 비동기이므로 uart_rx가 2FF로 동기화한다. 하강 에지(start)를 보면 반 비트 뒤 한 번 더 확인해 잡음을 거르고, 이후 한 비트 간격(DIV)마다 비트 중앙에서 LSB부터 8비트를 받는다. stop 비트 중앙이 1이면 rx_valid, 0이면 framing_error를 낸다. 에코 제어는 rx_valid 순간 송신기가 비어 있으면(tx_ready) 같은 바이트로 tx_valid를 한 클럭 올리고, 동시에 last_data를 갱신해 LED로 보낸다. uart_tx는 {stop, data, start} 10비트를 DIV마다 한 비트씩 오른쪽으로 밀어 uart_txd(F6)로 내보낸다.

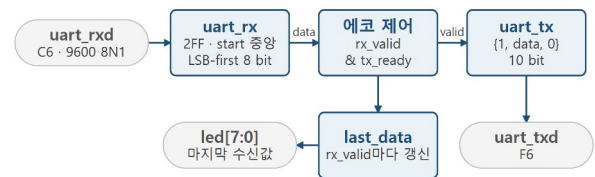


그림 25. LAB3-07 블록 흐름. 설계 top은 lab3_uart_echo.

파라미터 계산

항목	식	보드 (50 MHz)	TB
DIV	$(CLK_HZ + BAUD/2) / BAUD$	5,208클럭	8클럭
실제 baud	CLK_HZ / DIV	9,600.6 (+0.006 %)	100(가속)
1 bit	$DIV \times 20\text{ ns}$	104.16 μs	160 ns
1 frame	10 bit	1.0416 ms	1.6 μs
start 재확인	$DIV/2 - 1$	2,603클럭 후	3클럭 후

DIV는 반올림한 나눗셈이다. 오차 0.006 %는 한 프레임(10비트) 동안 샘플 위치가 비트 폭의 0.06 %만 밀리는 수준이다.

8N1 프레임과 수신기 동작 (예: 'A' = 0x41)

구간	비트 값	수신기 동작
idle	1	sync[1]=0이 될 때까지 대기
start	0	DIV/2 뒤 다시 0인지 확인
D0~D7	1,0,0,0,0,0,1,0	DIV마다 비트 중앙 샘플, LSB 먼저

구간	비트 값	수신기 동작
stop	1	중앙에서 1이면 valid, 0이면 framing_error

RTL·TB·XDC의 역할

파일	역할
src/uart.v	uart_rx(동기화, start 확인, 8비트 수신, stop 검사)와 uart_tx(10비트 시프트)
src/lab3_uart_echo.v	설계 top. DIV 계산, 에코 제어, last_data → led
sim/tb_uart_echo.sv	시뮬레이션 top. 독립 직렬 송수신 모델로 PC 역할
constraints/lab3_uart_echo.xdc	uart_rxd C6, uart_txd F6, LED 8핀, rst_p:uart_rxd false path

lab3_uart_echo.v의 에코 제어 부분이다.

```
always @(posedge clk_50mhz or posedge rst_p) begin
  if (rst_p) begin
    tx_valid <= 0; tx_data <= 0; last_data <= 0;
  end else begin
    tx_valid <= 0;
    if (rx_valid) begin
      last_data <= rx_data;
      if (tx_ready) begin
        tx_data <= rx_data;
        tx_valid <= 1;
      end
    end
  end
end
assign led = last_data;
```

GitHub:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/lab3_07_uart_echo/src/lab3_uart_echo.v#L28-L42

tx_ready가 0일 때 들어온 바이트는 LED에는 반영되지만 에코되지 않는다. 연속 입력에서는 이 조건이 중요하다(보드 확인 항목 참고).

TB 자극과 기대 결과

단계	TB 자극	기대 결과
1	rst_p 4클록 후 해제	uart_txd=1 (idle)
2	0x41 송신·수신 동시 진행	에코 0x41, led=0x41
3	0x5A	에코 0x5A, led=0x5A
4	0x0A	에코 0x0A, led=0x0A, framing error 없음

TB는 fork로 send_byte(PC 송신)와 receive_byte(PC 수신)를 동시에 돌린다. receive_byte는 uart_txd 하강 에지 뒤 1.5비트를 기다려 첫 데이터 비트 중앙부터 샘플한다.

Icarus PASS와 정상 파형

```
$ python tools/fpga_lab.py simulate
LAB3_UART_ECHO_PASS checks=3
sim/tb_uart_echo.sv:33: $finish called at 9970000 (1ps)
```

로그:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/logs/lab3_07_uart_echo_icarus_normal.txt

Icarus 종료 9970 ns, XSim 종료 9910 ns. 세 바이트 모두 에코와 LED 값이 맞았다. 두 시뮬레이터의 종료 시각이 60 ns 다른 이유는 다음과 같다. TB가 클록 상승 에지 직후 blocking 대입으로 uart_rxd를 바꾸는데, 같은 에지에서 DUT의 동기화 FF가 이전 값과 새 값 중 무엇을 읽는지가 시뮬레이터마다 다르기 때문이다(XSim은 110 ns, Icarus는 130 ns에 start 인식). 바이트당 1클록, 세 바이트 합계 3클록(60 ns) 차이이다. receive_byte가 uart_txd 하강 에지에 다시 맞추므로 PASS에는 영향이 없다.



그림 26. LAB3-07 정상 실행 전체 파형 (VaporView). uart_rxd 프레임 뒤에 uart_txd 에코가 이어진다.

수정 실험과 복구 결과

교안은 BAUD를 송수신 양쪽에서 같은 값으로 바꾸도록 한다. DUT의 BAUD를 100에서 200으로, TB 모델의 DIV를 8에서 4로 함께 바꿨다.

```
-localparam integer DIV=8;
+localparam integer DIV=4;
-lab3_uart_echo #(.CLK_HZ(800), .BAUD(100)) dut
+lab3_uart_echo #(.CLK_HZ(800), .BAUD(200)) dut
```

diff:

https://github.com/tae2yi/UOS_ece2/blob/090e0da/lab3/report/pre/mod/07_uart_echo/modification.diff

구분	내용
예상	DIV = (800+100)/200 = 4 → 1 bit 80 ns. 양쪽이 같으므로 PASS, 시간은 약 절반
수정 결과	LAB3_UART_ECHO_PASS checks=3, 종료 9970 → 5170 ns
복구 결과	DIV 8 → 1 bit 160 ns, LAB3_UART_ECHO_PASS checks=3 (9970 ns)

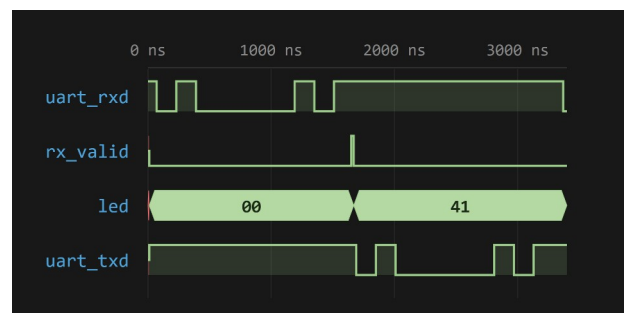


그림 27. 정상(BAUD=100): 0x41 한 프레임 1.6 μs, 뒤이어 에코.

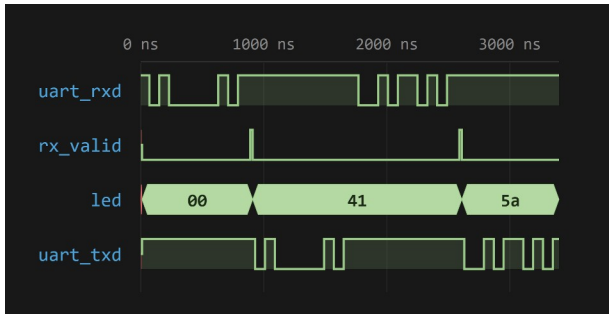


그림 28. 수정(BAUD=200): 같은 시간에 0x41·0x5A 두 바이트를 주고받음.

보드 기준 19,200 baud는 $DIV=2,604 (52.08 \mu s/bit)$ 다. PC 터미널도 같이 바뀌어야 하며, 한쪽만 바꾸면 샘플 위치가 어긋나 글자가 깨지거나 framing error가 난다.

보드에서 확인할 항목

확인 항목	예상 관찰
터미널 설정	9600, 8N1, 흐름 제어 없음, 로컬 에코 끄
한 글자	'A' 입력 → 'A' 표시, LED 0100_0001 (led[6], led[0])
제어 문자	Enter(0x0D) 등도 LED 값으로 확인
연속 입력	붙여넣기처럼 쉬지 않고 보내면 송신 완료와 다음 수신 사이 여유가 몇 클록뿐이라 일부 글자가 빠질 수 있음
배선	C6은 FPGA 입력(PC→FPGA). USB-UART 칩 방향과 GND 확인

5 비교와 결론

일곱 실험은 모두 50 MHz 한 클록과 카운터 기반 clock-enable 구조를 공유한다. 01·02는 비교기형 PWM, 03은 반주기 분주, 04는 enable 주기의 상태 순환, 05는 BCD 자리올림과 동적 스캔, 06은 tick 기반 명령 타이밍, 07은 비트 중앙 샘플링이다. 같은 카운터라도 끝 값이 무엇을 의미하는지(주기, 반주기, 간격, 비트)가 설계마다 다르며, 그 값을 파라미터 표로 계산했다.

실험	핵심 관찰	수정으로 바뀐 것
01	$threshold = 주기 \times level / 10$	단계 폭 10 % → 20 %
02	세 채널 독립, 같은 위상	R 초기 duty
03	$f = CLK / (2 \cdot 반주기)$	반주기 5 → 4
04	2비트 state ± 1 순환	step 간격 2배
05	BCD 자리올림, 1 kHz 스캔	1초 길이 2배
06	E 하강 에지에서 바이트 전달	0x52 → 0x53
07	비트 중앙 샘플, 반올림 DIV	bit 시간 절반

FAIL이 난 수정은 모두 TB 기대값을 그대로 둔 상태에서 변경된 동작과 기대가 처음 갈라지는 시점에 멈췄다. PASS가 난 04·07은 TB가 순서와 데이터만 검사하기 때문이며, 변화는 종료 시각과 파형 간격으로 확인했다. 보드 실험에서는 시뮬레이션이 다루지 않는 핀 배치, 극성, 전원·결선과 사람이 보고 듣는 결과(밝기, 색, 음, 회전, 표시, 터미널 문자)를 각 장의 표대로 확인한다.

참고 문헌

- [1] 이해리, 「LAB 3 · FPGA 응용회로 공통 예습」 및 LAB3-19-25 실험 교안(06.LAB3_*.pdf), 서울시립대학교 전자전기컴퓨터설계실험Ⅱ, 2026.
- [2] Hitachi, HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver 데이터시트.
- [3] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018.